EX 8 : IMAGE GENERATION USING VARIATIONAL AUTOENCODER (VAE)

AIM:

To Implement a Variational Autoencoder (VAE) to generate new images from a given dataset. Train the model to learn the latent representation of images and generate new samples from the learned distribution.

OBJECTIVES:

1. Understand the concept of generative models using latent space representations.
2. Implement encoder-decoder architecture with reparameterization.
3. Train a VAE on CelebA and generate new human face images.
4. Visualize generated samples from the latent space.

ALGORITHM:

1. Import the required libraries, including PyTorch, torchvision, and Matplotlib.

2. Configure the device, image size, batch size, latent dimension, number of epochs, and learning rate.

3. Load the CelebA dataset, preprocess the images, resize them to \(64 \times 64\), and select a subset of 500 images.

4. Define the encoder network using convolutional layers to extract image features and calculate the mean and log variance of the latent distribution.

5. Define the decoder network using transposed convolutional layers to reconstruct images from latent vectors.

6. Create the VAE model by combining the encoder, reparameterization function, and decoder.

7. Define the loss function using reconstruction loss and KL divergence to measure image reconstruction error and latent distribution difference.

8. Initialize the VAE model and Adam optimizer.

9. Train the model for five epochs by calculating the loss, performing backpropagation, and updating the model parameters.

10. Display the average training loss after each epoch.

11. Generate random latent vectors and pass them through the decoder to create new images.

12. Denormalize and display the generated images using Matplotlib.

SOURCE CODE:

```
# Step 1: Import Required Libraries

import torch
import torch.nn as nn
import torch.nn.functional as F
import torchvision
from torchvision import datasets, transforms
from torch.utils.data import DataLoader, Subset
import matplotlib.pyplot as plt

# Step 2: Configure Parameters and Load Dataset

device = torch.device('cuda' if torch.cuda.is_available() else 'cpu')

image_size = 64
batch_size = 128
latent_dim = 100
num_epochs = 5
learning_rate = 1e-3
transform = transforms.Compose([
    transforms.Resize((image_size, image_size)),
    transforms.ToTensor(),
    transforms.Normalize((0.5, 0.5, 0.5),
                         (0.5, 0.5, 0.5))
])

# Download CIFAR-10 instead of CelebA
dataset = datasets.CIFAR10(
    root='data',
    train=True,
    download=True,
    transform=transform
)

# Use 500 images for faster training
dataset = Subset(dataset, range(500))

dataloader = DataLoader(
    dataset,
    batch_size=batch_size,
    shuffle=True
)
```

```
print("Device:", device)
print("Dataset size:", len(dataset))

# Step 3: Define Encoder Network

class Encoder(nn.Module):

    def __init__(self, latent_dim):
        super(Encoder, self).__init__()
        self.conv = nn.Sequential(
            nn.Conv2d(3, 64, 4, 2, 1),
            nn.ReLU(),
            nn.Conv2d(64, 128, 4, 2, 1),
            nn.BatchNorm2d(128),
            nn.ReLU(),
            nn.Conv2d(128, 256, 4, 2, 1),
            nn.BatchNorm2d(256),
            nn.ReLU(),
            nn.Conv2d(256, 512, 4, 2, 1),
            nn.BatchNorm2d(512),
            nn.ReLU()
        )

        self.fc_mu = nn.Linear(512 * 4 * 4, latent_dim)
        self.fc_logvar = nn.Linear(512 * 4 * 4, latent_dim)
    def forward(self, x):
        x = self.conv(x)
        x = x.view(x.size(0), -1)
        mu = self.fc_mu(x)
        logvar = self.fc_logvar(x)
        return mu, logvar

# Step 4: Define Decoder Network

class Decoder(nn.Module):
    def __init__(self, latent_dim):
        super(Decoder, self).__init__()
        self.fc = nn.Linear(latent_dim, 512 * 4 * 4)
        self.deconv = nn.Sequential(
            nn.ConvTranspose2d(512, 256, 4, 2, 1),
            nn.BatchNorm2d(256),
            nn.ReLU(),
```

```
            nn.ConvTranspose2d(256, 128, 4, 2, 1),
            nn.BatchNorm2d(128),
            nn.ReLU(),
            nn.ConvTranspose2d(128, 64, 4, 2, 1),
            nn.BatchNorm2d(64),
            nn.ReLU(),
            nn.ConvTranspose2d(64, 3, 4, 2, 1),
            nn.Tanh()
        )
    def forward(self, z):
        x = self.fc(z)
        x = x.view(x.size(0), 512, 4, 4)
        return self.deconv(x)

# Step 5: Define the VAE Model

class VAE(nn.Module):
    def __init__(self, latent_dim):
        super(VAE, self).__init__()
        self.encoder = Encoder(latent_dim)
        self.decoder = Decoder(latent_dim)
    def reparameterize(self, mu, logvar):
        std = torch.exp(0.5 * logvar)
        eps = torch.randn_like(std)
        return mu + eps * std
    def forward(self, x):
        mu, logvar = self.encoder(x)
        z = self.reparameterize(mu, logvar)
        return self.decoder(z), mu, logvar

# Step 6: Define Loss Function

def vae_loss(recon_x, x, mu, logvar):
    recon_loss = F.mse_loss(
        recon_x, x, reduction='sum'
    )
    kl_div = -0.5 * torch.sum(
        1 + logvar - mu.pow(2) - logvar.exp()
    )
    return recon_loss + kl_div

# Step 7: Train the Model and Generate Images
```

```
model = VAE(latent_dim).to(device)
optimizer = torch.optim.Adam(
    model.parameters(),
    lr=learning_rate
)
for epoch in range(num_epochs):
    model.train()
    total_loss = 0
    for images, _ in dataloader:
        images = images.to(device)
        # Forward pass
        recon, mu, logvar = model(images)

        # Calculate loss
        loss = vae_loss(recon, images, mu, logvar)

        # Backpropagation
        optimizer.zero_grad()
        loss.backward()
        optimizer.step()
        total_loss += loss.item()

    print(
        f"Epoch [{epoch + 1}/{num_epochs}], "
        f"Loss: {total_loss / len(dataloader.dataset):.4f}"
    )

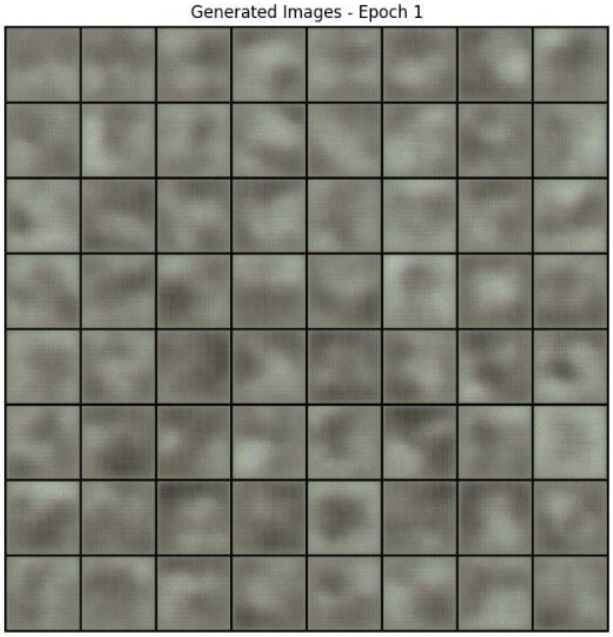
    # Generate and visualize images
    model.eval()
    with torch.no_grad():
        z = torch.randn(64, latent_dim).to(device)
        sample_images = model.decoder(z).cpu()
        # De-normalize images
        sample_images = sample_images * 0.5 + 0.5
        grid = torchvision.utils.make_grid(
            sample_images,
            nrow=8
        )
        plt.figure(figsize=(8, 8))
        plt.imshow(grid.permute(1, 2, 0).clamp(0, 1))
        plt.axis('off')
        plt.title(f"Generated Images - Epoch {epoch + 1}")
        plt.show()
```

print("VAE training completed successfully!")

OUTPUT:

```
Device: cpu
100%|██████████| 170M/170M [44:54<00:00, 63.3kB/s]
Dataset size: 2000
Image size: 64 x 64
Epoch [1/10] Total Loss: 8320.30 | Reconstruction Loss: 8280.01 | KL Divergence: 40.30
```

Generated Images - Epoch 1



```
Epoch [2/10] Total Loss: 7646.64 | Reconstruction Loss: 7577.48 | KL Divergence: 69.16
```

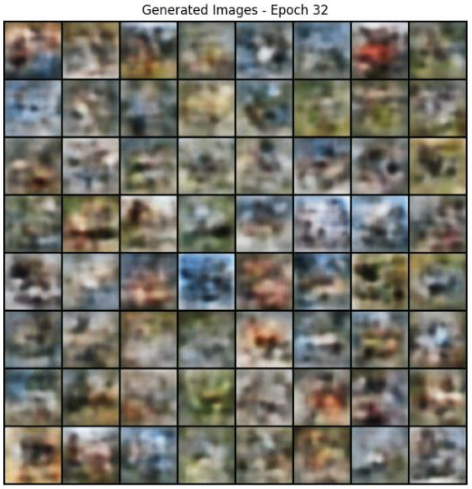
```
Epoch [11/50] Total Loss: 7298.59 | Reconstruction Loss: 7193.99 | KL Divergence: 104.59
Epoch [12/50] Total Loss: 7286.05 | Reconstruction Loss: 7181.41 | KL Divergence: 104.64
```

Generated Images - Epoch 12



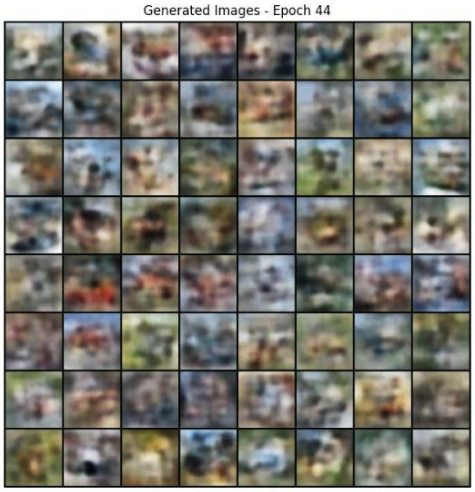
```
Epoch [13/50] Total Loss: 7274.97 | Reconstruction Loss: 7170.62 | KL Divergence: 104.35
Epoch [14/50] Total Loss: 7267.96 | Reconstruction Loss: 7163.78 | KL Divergence: 104.18
```

Generated Images - Epoch 32



```
Epoch [33/50] Total Loss: 7181.32 | Reconstruction Loss: 7075.90 | KL Divergence: 105.42
Epoch [34/50] Total Loss: 7186.51 | Reconstruction Loss: 7080.30 | KL Divergence: 106.21
```

Generated Images - Epoch 44



```
Epoch [45/50] Total Loss: 7162.82 | Reconstruction Loss: 7056.14 | KL Divergence: 106.69
Epoch [46/50] Total Loss: 7167.33 | Reconstruction Loss: 7060.62 | KL Divergence: 106.71
```

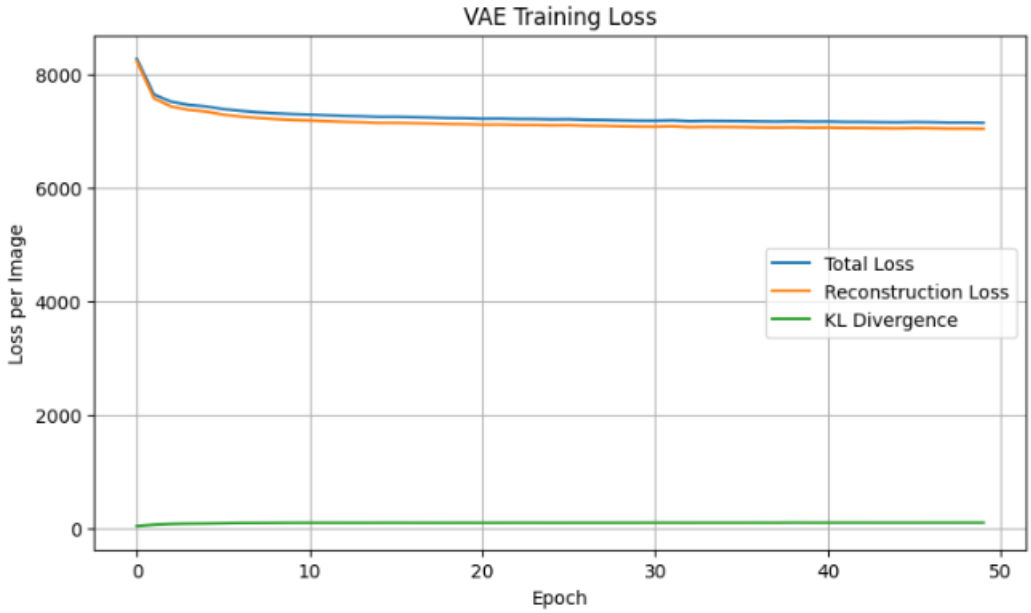
Original Images

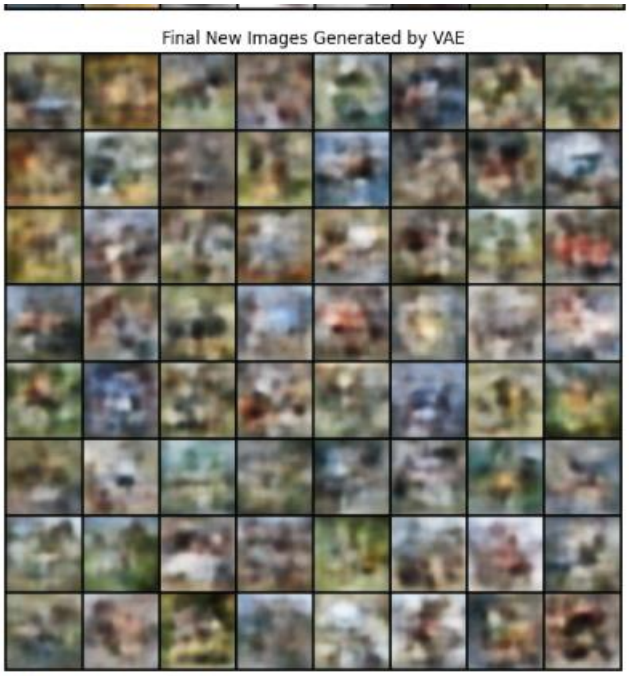


Reconstructed Images



```
Training completed successfully!
```






```
Model saved as vae_model.pth

RESULT:
The VAE learned latent representations.
Original and reconstructed images were displayed.
New images were generated by sampling the latent space.
```

RESULT:

In this experiment, a Variational Autoencoder was implemented and trained on a subset of the CelebA dataset. The model effectively learned to encode facial features into a latent space and generate realistic synthetic faces from random samples in that space. This showcases the generative power of deep neural architectures.